

Redis Configuration Guide

Critical: Eviction Policy Configuration

Current Issue

Your Redis instance is configured with `maxmemory-policy volatile-lru`, which can cause data loss in BullMQ queues and cache. The system logs show:

```
IMPORTANT! Eviction policy is volatile-lru. It should be "noeviction"
```

Why This Matters

- **volatile-lru**: Redis evicts keys with an expiration set when memory limit is reached
- **noeviction**: Redis returns errors when memory limit is reached, preventing data loss
- BullMQ stores critical job data that should **never** be evicted automatically

How to Fix

Option 1: Update Redis Configuration File (Recommended)

1. Find your Redis configuration file:

```
# Common locations:
/etc/redis/redis.conf
/usr/local/etc/redis/redis.conf
```

2. Edit the configuration:

```
sudo nano /etc/redis/redis.conf
```

3. Find and update these lines:

```
# Change this:
maxmemory-policy volatile-lru

# To this:
maxmemory-policy noeviction
```

4. Restart Redis:

```
sudo systemctl restart redis
# or
sudo service redis-server restart
```

Option 2: Update via Redis CLI (Temporary until restart)

```
redis-cli
> CONFIG SET maxmemory-policy noeviction
> CONFIG REWRITE # Makes it persistent (if Redis has write access to config)
> exit
```

Option 3: For Docker/Containerized Redis

Add to your `docker-compose.yml` or `docker run` command:

```
redis:
  image: redis:7-alpine
  command: redis-server --maxmemory-policy noeviction
```

Option 4: For Managed Redis (AWS ElastiCache, DigitalOcean, etc.)

1. Go to your Redis instance settings in the cloud provider dashboard
2. Find "Parameter Group" or "Configuration" settings
3. Change `maxmemory-policy` to `noeviction`
4. Apply changes (may require restart)

Verify the Fix

```
redis-cli CONFIG GET maxmemory-policy
```

Expected output:

```
1) "maxmemory-policy"  
2) "noeviction"
```

Additional Recommendations

1. Set appropriate maxmemory:

```
maxmemory 512mb # Adjust based on your server RAM
```

2. For KVM1 (4GB RAM server), recommended Redis settings:

```
maxmemory 1gb  
maxmemory-policy noeviction  
save 900 1  
save 300 10  
save 60 10000
```

3. Monitor Redis memory usage:

```
redis-cli INFO memory
```

Impact After Fix

- ☒ No more automatic eviction of queue jobs
- ☒ System will properly handle out-of-memory scenarios
- ☒ Queue jobs will be processed reliably
- ⚠ Applications must handle Redis OOM errors gracefully

Connection Pool Optimization

Changes Made

- Removed 3 QueueEvents to reduce Redis connections by 3
- Optimized connection settings in queue configuration
- Current connection count: ~12 connections (down from ~15)

Connection Breakdown

- 5 Queues × 2 connections each = 10
- 2 Workers × 2 connections each = 4 (in separate process)
- 1 Redis client = 1
- **Total: ~15 connections** (12 from backend + 3-4 from workers)

Monitor Connections

```
redis-cli CLIENT LIST | grep -c "addr"
```

If Connection Issues Persist

1. Increase Redis max clients:

```
redis-cli CONFIG SET maxclients 500  
redis-cli CONFIG REWRITE
```

2. Check current limit:

```
redis-cli CONFIG GET maxclients
```

3. Or in redis.conf:

```
maxclients 500
```

Troubleshooting

Server Keeps Restarting

If your backend is restarting every 2-3 seconds:

1. Check PM2 memory limit:

```
pm2 info gema-backend
```

2. Increase if needed (in `ecosystem.config.js`):

```
max_memory_restart: '750M' // Increase from 500M
```

3. Check for unhandled promise rejections in logs:

```
pm2 logs gema-backend --err
```

Redis Connection Errors

If you see `ECONNREFUSED 127.0.0.1:6379`:

1. Verify Redis is running:

```
sudo systemctl status redis
```

2. Check Redis is listening:

```
sudo netstat -tlnp | grep 6379
```

3. Test connection:

```
redis-cli ping # Should return "PONG"
```

Summary of Fixes Applied

1. ☒ **Removed duplicate `userId` index in `Vendor` model** - Fixed Mongoose warning
2. ☒ **Disabled `QueueEvents`** - Reduced Redis connections by 3
3. ☒ **Optimized queue configuration** - Better connection pooling
4. ☒ **Redis eviction policy** - Requires server-side configuration (this document)

Next Steps

1. Update Redis eviction policy (see above)
2. Monitor server stability after changes
3. Watch Redis memory usage
4. Consider upgrading Redis memory if needed

Questions?

If issues persist after applying these fixes, check

- Application logs: `pm2 logs gema-backend`
- Redis logs: `sudo tail -f /var/log/redis/redis-server.log`
- System resources: `htop` or `free -h`